Functions as Objects
Generators, generator expressions
Lambda, maps, filters
Profiling

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 6

Functions are objects

Try typing the following in your command shell/Jupyter or PyCharm:

- The variable **function_obj** is pointing to the function **some_function**.
- Can now use the variable **function_obj** as though it was the function itself

```
def some_function():
     print("Something")

function_obj = some_function
print(function_obj) #<function some_function at 0x007B67C0>
print(function_obj.__name__) #some_function
print(function_obj.__class__) #<class 'function'>
print(type(function_obj)) #<class 'function'>
function_obj() #Something
```

Dictionaries and Functions

Functions are treated in an OOP manner, that is to say, they are objects.

Everyone has already been using functions as objects!

- Some of you had errors in your code where you forgot to mention the parenthesis () with a function name but the compiler complained as if something was wrong with an object
- Some of you have tried putting functions in dictionaries and mapped them to menu choices.

Let's look at some code to refresh our memory

function_objects.py

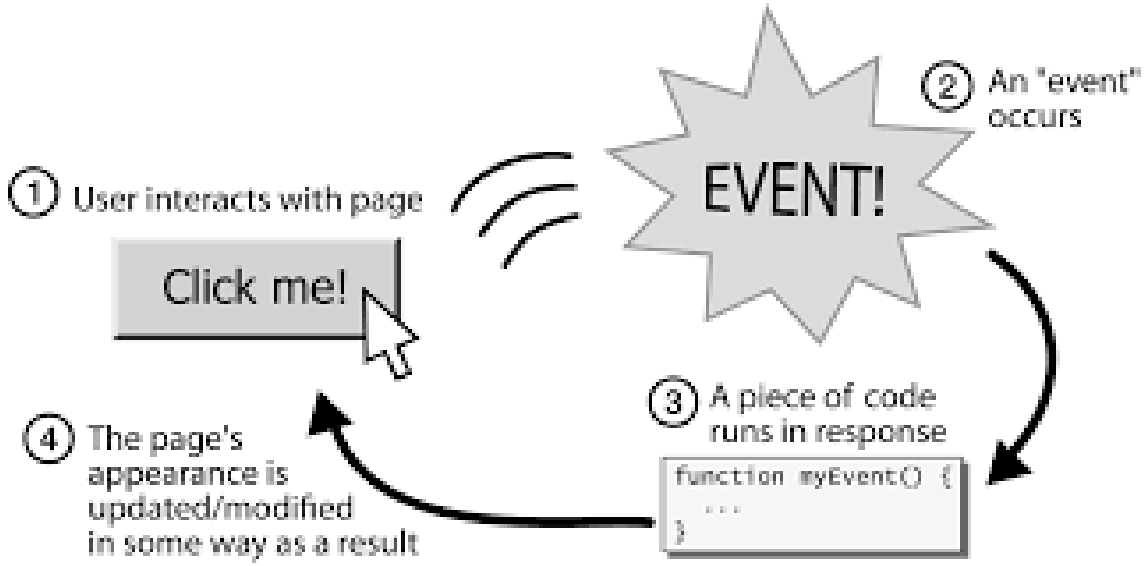
```
input_map = {
    1: sum,
    2: subtract,
    3: multiply,
    4: divide,
}
```

Functions as Objects: **Event Driven Programming**

1. We can store functions as objects
2. **Pass them** around via arguments
3. **Call them later** in the future when some condition has been met.

These functions that are passed to be called back later are known as **callbacks**.

That is the essence of event-driven programming.

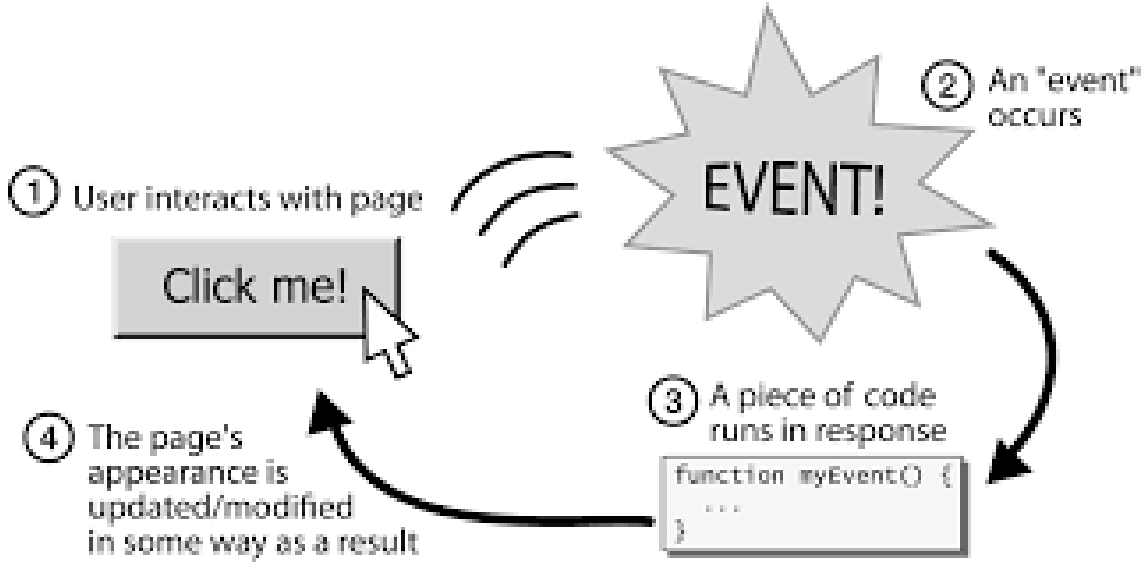


Functions as Objects: **Event Driven Programming**

Functions that are meant to handle an event such as a mouse or button click are known as **Event Handlers**

<u>Event Handlers are just another type of callback</u>. You could fire multiple event handlers when an event occurs.

That is the essence of event-driven programming.



Event Driven Programming

This is extremely simple in Python.

We don’t need to worry about advanced constructs in other languages like delegates

We don’t need an artificial object to wrap around 1 function so we can pass it around.

Things are much simpler in Python since functions are objects themselves!

Let’s look at an example!

(refer to the callback_example.py sample code)

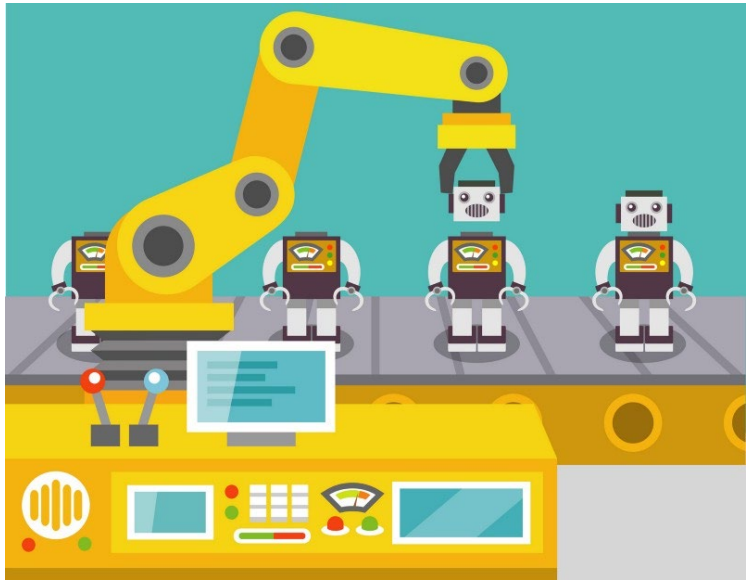
Generators

Generators

Generators are a special kind of iterator.

They operate over a sequence of objects / iterables.

Generators use a `yield` statement to return the next element in a sequence.



Generators

So for a moment, let's forget about generators and yield statements. Let's look at ranges instead.

Ranges are awesome because:

- They save memory
- They are easy to read
- They provide all the advantages of being an iterable.

Generators

```
my_range = range(1000)
a_list = [i for i in range(1000)]

print(my_range) #range(0,1000)
print(a_list) #[0,1,2, . . ., 997,998,999]

print(f"Range Size in bytes: {sys.getsizeof(my_range)}") #Range Size in
bytes: 48
print(f"List Size in bytes: {sys.getsizeof(a_list)}") #List Size in
bytes: 8856
```

why_generators.py*

Generators

Ranges aren't exactly like generators but the concept is the same

Generators are special constructs and a special type of iterator that iterates over a sequence while saving memory.

Think of generators as objects that iterate over a sequence.

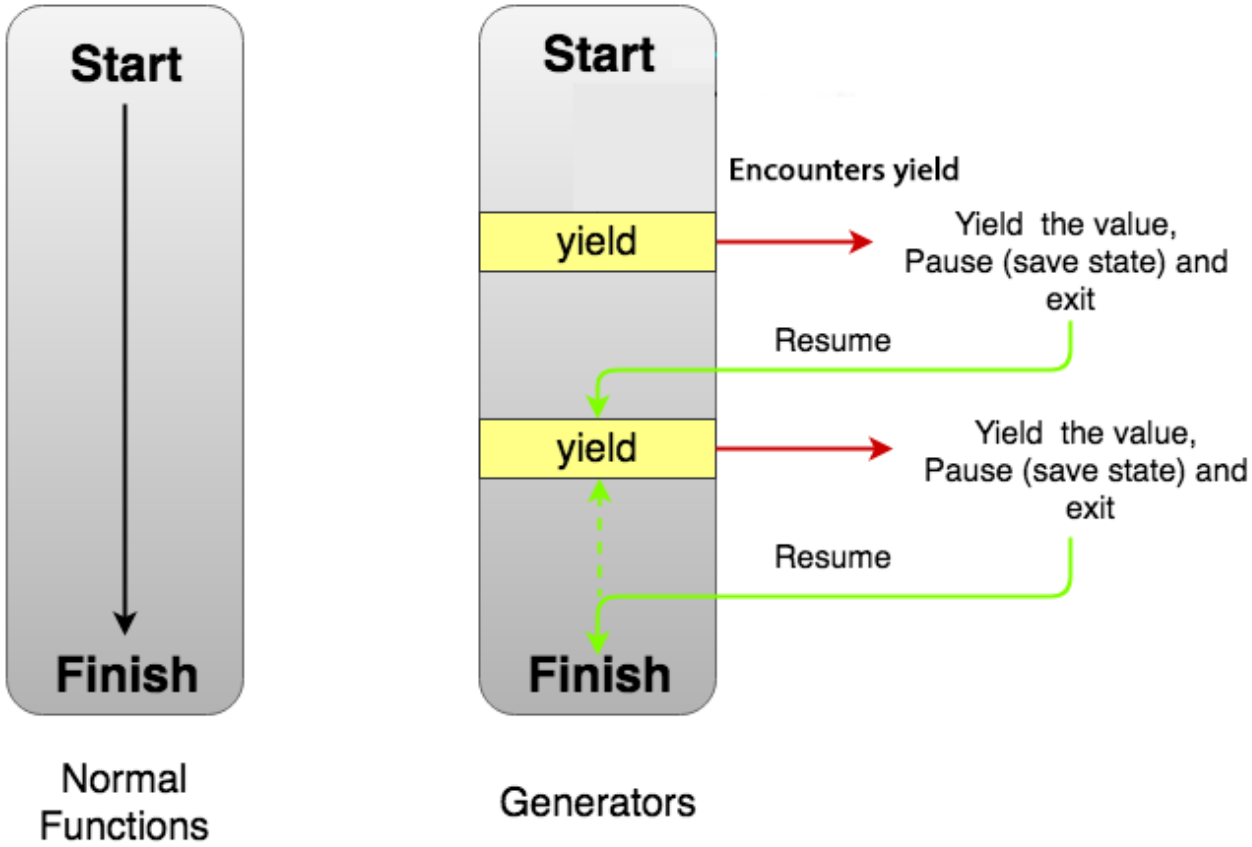
However instead of creating and storing the entire sequence at once, they **only contain the code to *generate* the next object in a sequence**

Generator function

How do we create a generator? Just have a yield statement in a function

As soon as a function has one or more yield statements, the function is now considered a generator

```
def my_function():
    n = 0
    #some code
    yield n #returns n and pauses. Execution continues from next line later
    #some more code
    yield n
```



Generators - Yield

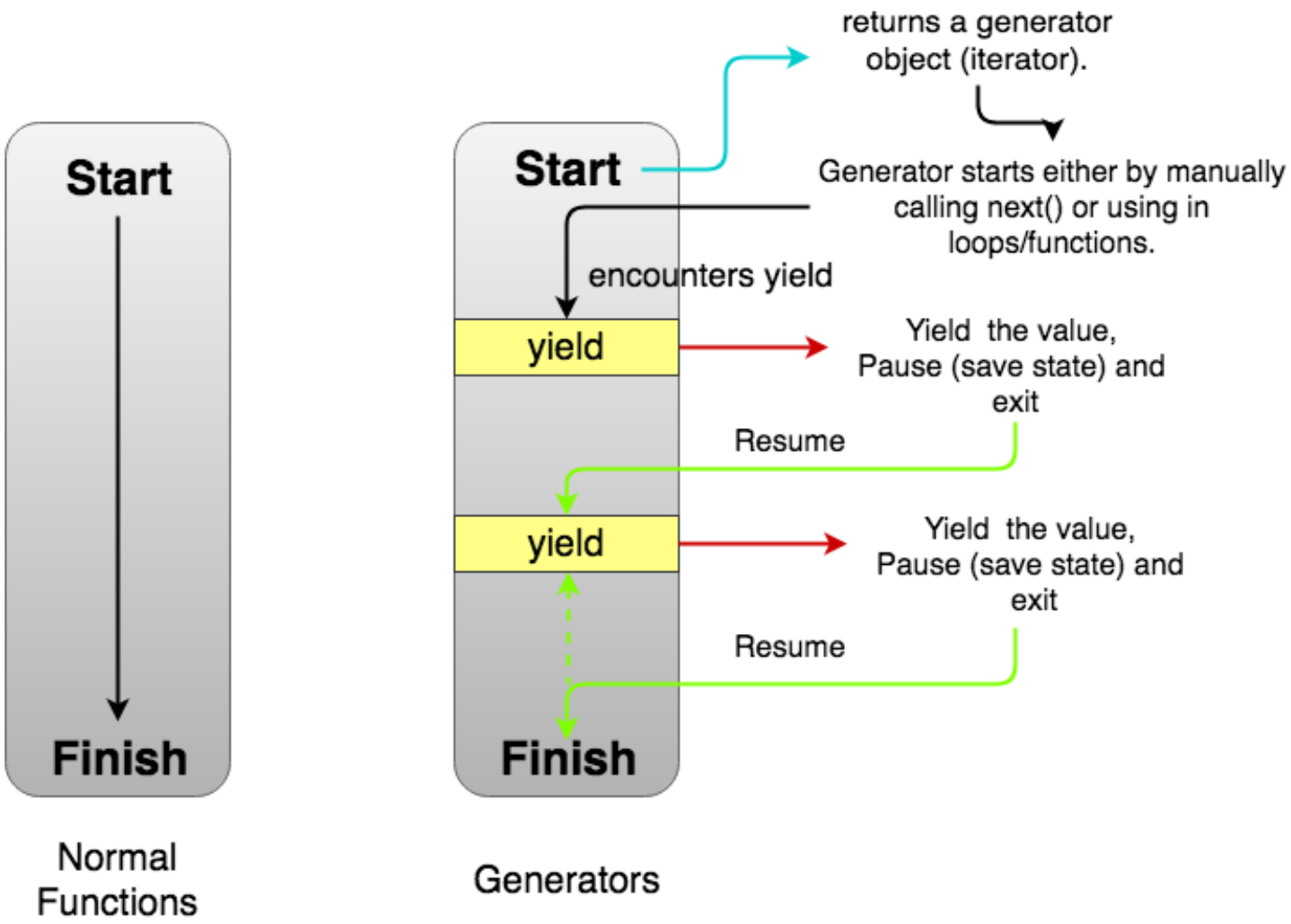
The `yield` statement is a special statement.

Think of it like a return statement, but instead of returning from a function it **returns the value and remembers its place in the function**.

The next time the function is called, it **resumes execution by executing the next line after the yield statement.**

You are temporarily *'yielding'* the flow of control back to the entity that called the function.

Generator Functions



When python sees a generator function with a yield statement it **wraps it up in an object**.

This **object functions like an iterator**.

Every time `next(generator_object)` is called, the generator function executes. This happens behind-the-scenes.

Subsequent calls to next, continue from the instruction after the last yield statement.

generator_example.py

Generators

Let's go back to our log file example from a while ago.

How do we filter out all the lines with the [Warning] tag in it **without creating a temporary variable that takes up memory**?

```
user@host> file show /var/log/processes Feb 22 08:58:24 router1 snmpd[359]:
%DAEMON-3-SNMPD_TRAP_WARM_START: trap_generate_warm: SNMP trap: [Warning]
warm start
Feb 22 20:35:07 router1 snmpd[359]:
%DAEMON-6-SNMPD_THROTTLE_QUEUE_DRAINED: [Warning]
trap_throttle_timer_handler: cleared all throttled traps
Feb 23 07:34:56 router1 snmpd[359]:
%DAEMON-3-SNMPD_TRAP_WARM_START: trap_generate_warm: SNMP trap: warm start
Feb 23 07:38:19 router1 snmpd[359]:
%DAEMON-2-SNMPD_TRAP_COLD_START: trap_generate_cold: [Warning] SNMP trap:
cold start
```

concrete_generator.py

https://en.wikipedia.org/wiki/Fibonacci_number

Fibonacci sequence with generators

In mathematics, the **Fibonacci numbers**, commonly denoted F_n, form a sequence, called the **Fibonacci sequence**, such that each number is the sum of the two preceding ones, starting from 0 and 1.

0, 1

0, 1, 1

0, 1, 1, 2

0, 1, 1, 2, 3

0, 1, 1, 2, 3, 5

0, 1, 1, 2, 3, 5, 8

0, 1, 1, 2, 3, 5, 8, 13

0, 1, 1, 2, 3, 5, 8, 13, 21

0, 1, 1, 2, 3, 5, 8, 13, 21, 34…

Fibonacci sequence with generators

1. Imagine creating a **function** that returns the next number in the Fibonacci sequence every time it's called

This might require some static or global variables to keep track of the numbers.

2. Or we might instead write the code in a **class**
   - Classes can keep track of the numbers with instance variables
   - Can write a get_next_fib() function to get the next number

3. Or we can use **generators**

Fibonacci sequence with generators

Infinite Fibonacci generator. Calling next(fib) will give the next number in Fibonacci sequence indefinitely

```
def fibonacci_sequence():
    current = 0
    previous = 1
    while True:
        yield current
        previous, current = current, previous + current

#call generator with for loop
fib = fibonacci_sequence()
for x in range(0,100):
    print(next(fib), end=", ")
```

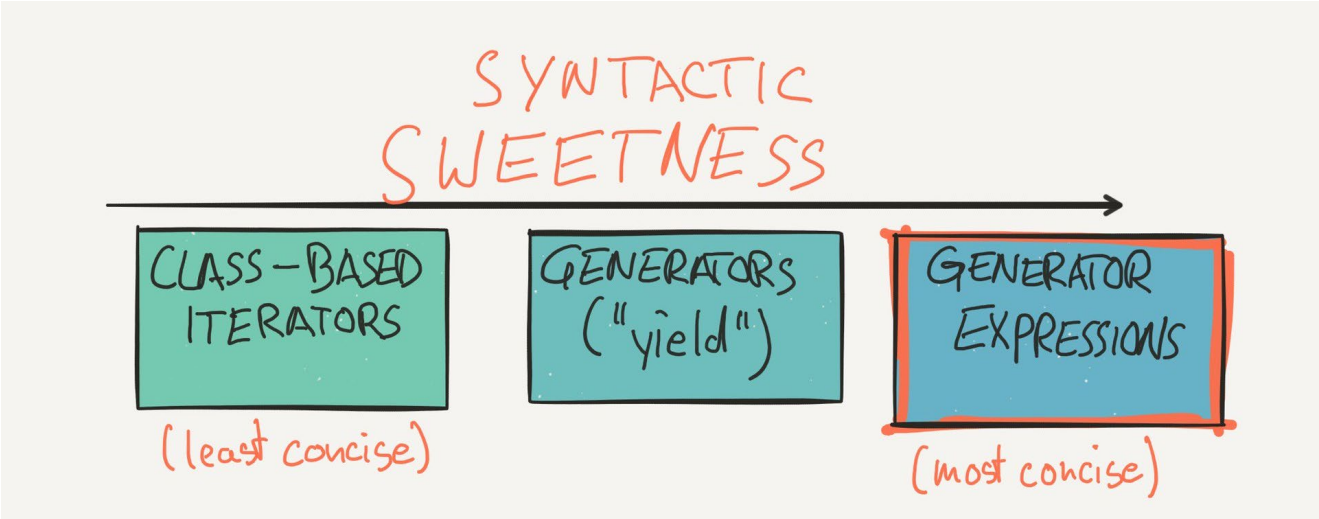
Output:
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 … 7778742049]

generator_examples.py

Generator Expressions

What if I told you that there was **an even better** way of using generators!

What if I told you that you could add even **more syntactic sugar** to your code!



Generator Expressions

Similar to a list comprehension.

Syntactically it is exactly the same with the exception that we now use Parentheses – **()** instead of Square brackets – []

These are not as flexible as generator functions, but are very ***Clean*** .

- Not as flexible because these generators **can not be re-used**

```
( f(x) for x in some_iterable if condition )
```

Generator Expressions

Say we want to

- Access all the even numbers from 0 – 100 (excluding 100)
- Multiply it by 5

By using a generator instead of a list comprehension we save memory.

A range uses generators in its implementation. But all the rules of a generator don't apply to ranges.

The **ok** way of doing this:

```
gen_exp = ( x * 5 for x in range(100) if x % 2 == 0)
for i in gen_exp:
    print(i)
```

Generator Expressions

```
gen_exp = ( x * 5 for x in range(100) if x % 2 == 0)
for i in gen_exp:
    print(i)
```

The code above is only ‘ok’ because it is **not re-usable**

Once we iterate over gen_exp, iterating over it again will produce no output

Generator Expressions

The **good** way of doing this, would be to use the **yield from** syntax:

```
def gen_even_multiples(multiplier, start, stop):
    yield from (x * multiplier for x in range(start, stop) if x % 2 == 0)


for result in gen_even_multiples(5, 0, 100):
    print(result)
```

Notice how we wrapped the *generator expression* in a **generator function**

Keep in mind, when calling generators within generators, use the **yield from** keyword to maintain the yield functionality

Generator Expressions

Remember our previous log file parser? A function with **multiple lines**

```
def warning_filter_generator(file_object):
    for line in file_object:
        if '[Warning]' in line:
            yield line


with open("log_file.txt", mode='r', encoding='utf-8') as log_file:
    filter = warning_filter_generator(log_file)
    for warning_line in filter:
        print(warning_line)
```

concrete_generator.py*

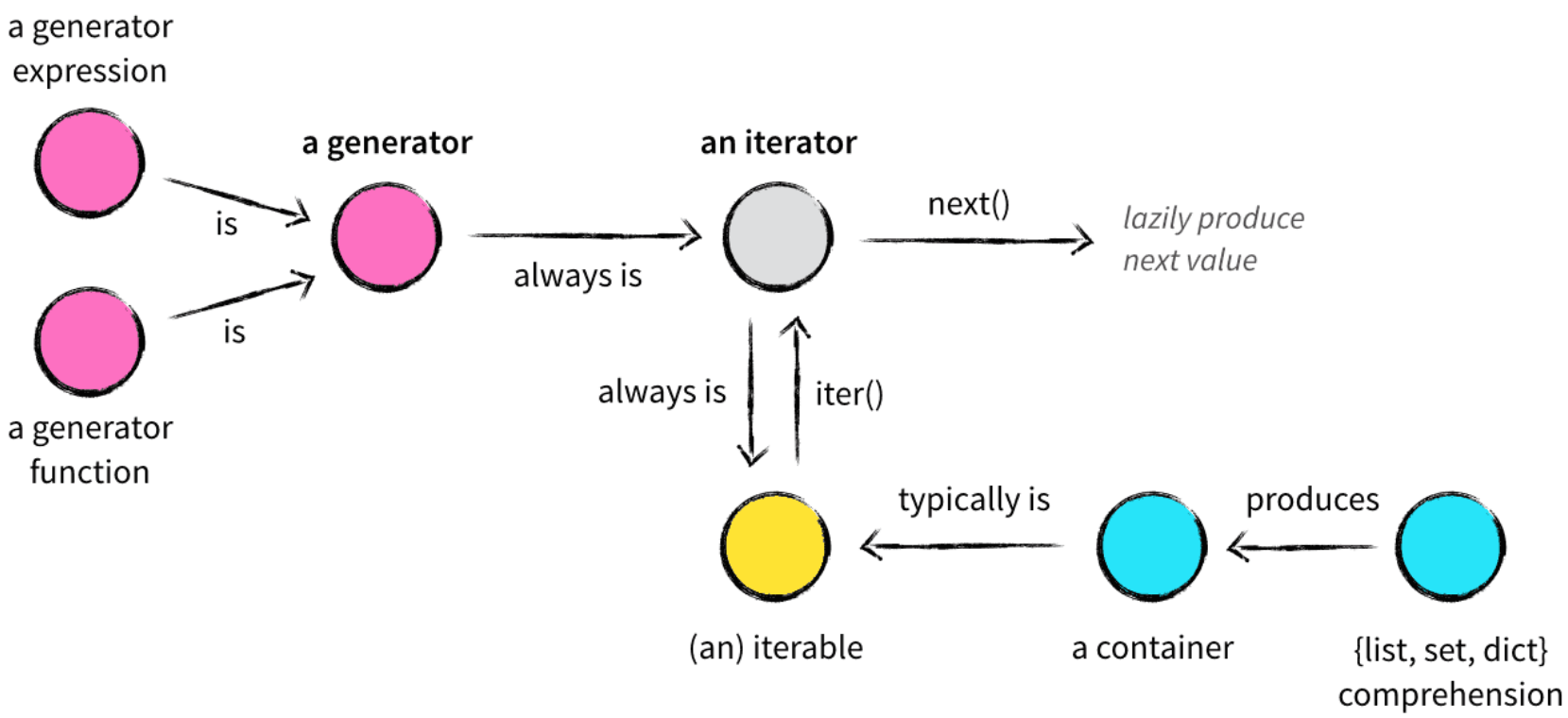
Generator Expressions

We can make our previous log file parser ***even better*** now that we know about *generator expressions*:

```
def warning_filter_generator(file_object):
    #one line with generator expression!
    yield from (line for line in file_object if '[Warning]' in line)


with open("log_file.txt", mode='r', encoding='utf-8') as log_file:
    filter = warning_filter_generator(log_file)
    for warning_line in filter:
        print(warning_line)
```

generator_examples.py*

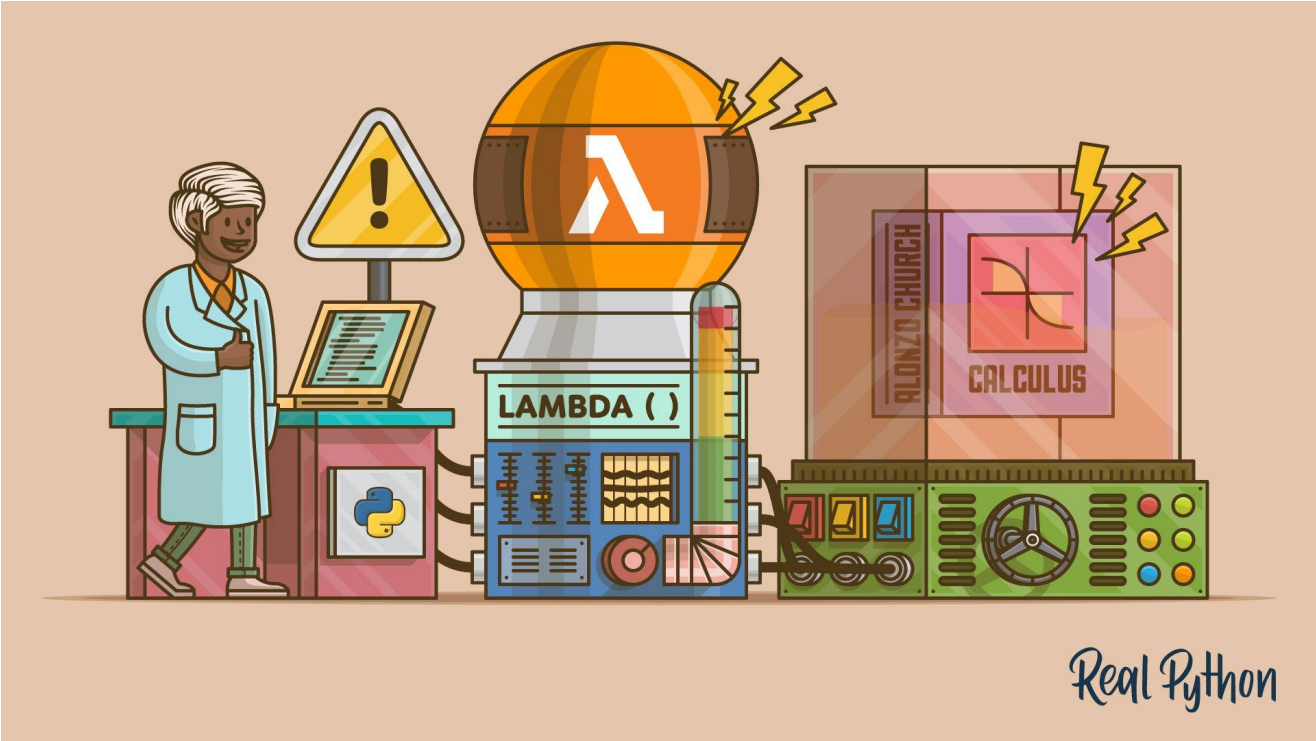
Generators - Summary



Generators - Summary

- Generators are a special kind of iterator
- They make use of **yield statements**
- They save memory by not actually reserving a contiguous chunk of memory for an iterable.
- They should be used whenever you need to work on an iterable, one element at a time, but don't want to actually change the iterable itself or store the result in memory
- Extremely useful when working with complex objects and Big Data. (Who knew big data takes up a lot of memory?)

Lambda Functions



Lambda Functions – Anonymous Functions

Lambda Functions are also known as Anonymous Functions

These are functions without a name

Think of defining a temporary function which has a small scope.

Regular function:

```
def my_func(parameters):
    expression
```

Syntax:

```
lambda parameters: expression
```

Lambda Functions – Anonymous Functions

Regular function:

```
def my_func(x):
    return x * 5
```

Lambda function

```
lambda x: x * 5
```

This is a lambda function that takes 1 argument `x` and **`returns`** `x * 5`

Lambda Functions - example

Can store lambdas into a **variable**.

Can use that **variable** as if it was the lambda

```
multiple_of_five = lambda x: x * 5
print(multiple_of_five(10)) #50
```

<u>Lambda functions aren't multi-line!</u>

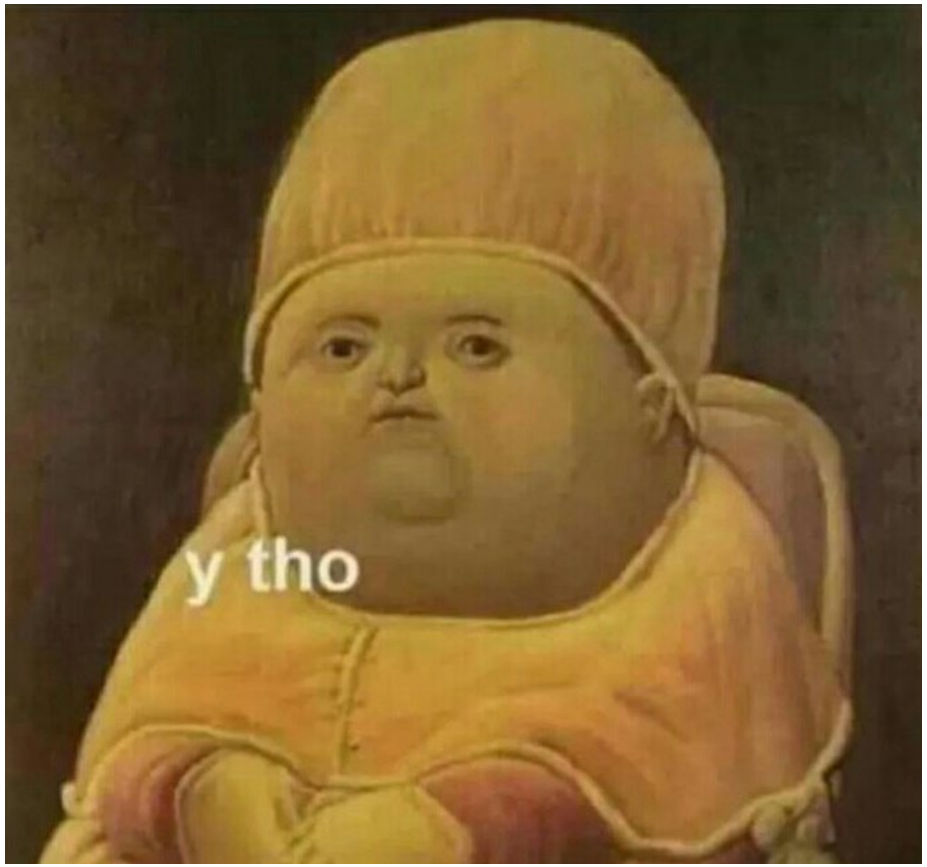
When do we use lambda functions?

- To define callbacks.
- Asynchronous code
  i.e. code that does not run sequentially.

For example, sending an HTTP request and then executing some code when the response is received.

The send and response are considered asynchronous functions. They don't run in sequence and there may be a wait time.

- In list comprehensions/generators



Lambda Functions in use.

In a system far, far away….

We have a function:

```
def send_data_to_backend(protocol, data, response_callback):
      # do fancy web code here. Unknown time for operation to finish
      # execute response_callback when done
      response_callback("response message")
```

Somewhere else in our system:

```
def send_analytics(user_data):
      send_data_to_backend("http", user_data, lambda response: print(response))
```

Can use lambda functions to avoid writing a new function definition.

Can create an anonymous function and pass it as an argument. At some point it will get executed (once the response is received).

Lambda Functions in use.

Challenge Time!!

Step 1: Write a lambda function that takes an argument and returns the square root of the argument.

Step 2: Save this in a variable.

Step 3: Now use this lambda function in a list comprehension to create a new list which contains the square roots of each number in the list [1, 2, 3, 4, 5].

Step 4: Print the list

Lambda Functions in use.

Challenge Time!!

Step 1: Write a lambda function that takes an argument and returns the square root of the argument.

Step 2: Save this in a variable.

Step 3: Now use this lambda function in a list comprehension to create a new list which contains the square roots of each number in the list [1, 2, 3, 4, 5].

Step 4: Print the list

```
import math

square_root = lambda x: math.sqrt(x)
list = [square_root(x) for x in range(1,6)]

print(list)
```

Map

map(function, sequence)

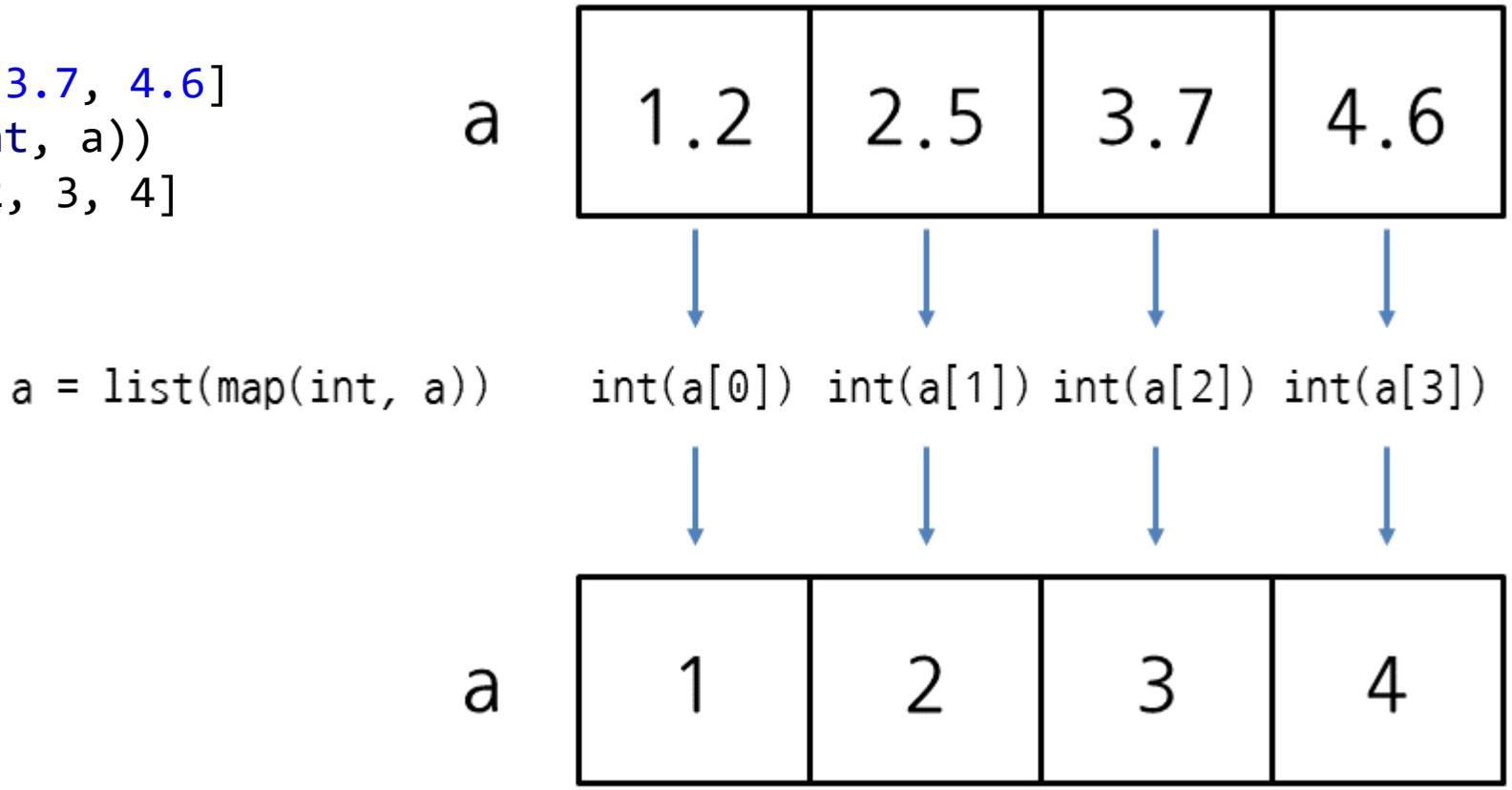
- An alternative to list comprehensions.
- The map function takes 2 arguments. A function and an iterable.
  - **map(*function*, *iterable*)**
- The map applies the function to each element of the iterable and returns a map object.
- This map object is an iterator that represents a new sequence containing the results of applying the function to each element in the iterable

To sum it up:

You are *mapping* a function to each element in an iterable and returning the resultant sequence via an iterator.

map(int, [1.2, 2.5, 3.7, 4.6])

```
a = [1.2, 2.5, 3.7, 4.6]
a = list(map(int, a))
print(a) #[1, 2, 3, 4]
```



Map(function, sequence): Print first names only

```
full_names = [
    "Ross Geller",
    "Rachel Green",
    "Phoebe Buffay",
    "Chandler Bing",
    "Monica Geller",
    "Joey Tribbiani"]

first_name_map = map(lambda x: x.split()[0], full_names)
print(type(first_name_map))
print(list(first_name_map))
```

Map the function x.split()[0] to every element in the full_names list

Map(function, sequence): Print first names only

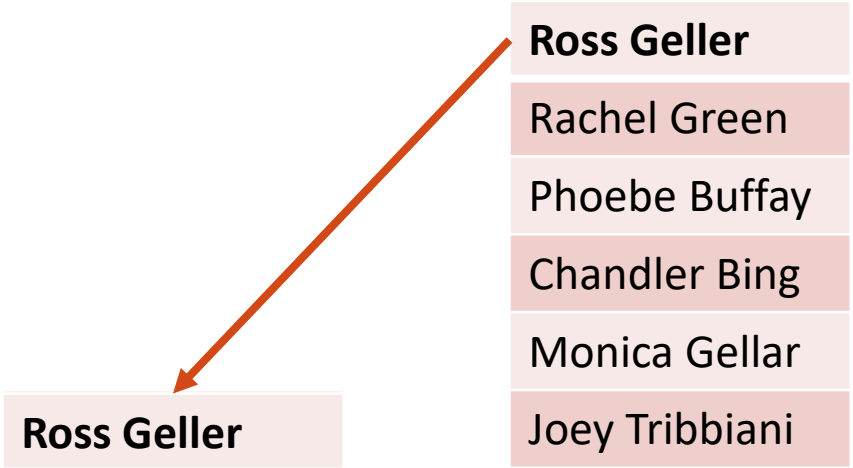
```
full_names = [
    "Ross Geller",
    "Rachel Green",
    "Phoebe Buffay",
    "Chandler Bing",
    "Monica Geller",
    "Joey Tribbiani"]

first_name_map = map(lambda x: x.split()[0], full_names)
print(type(first_name_map))
print(list(first_name_map))
```

Ross Geller
Rachel Green
Phoebe Buffay
Chandler Bing
Monica Gellar
Joey Tribbiani

Have the full list of first and last names

Map(function, sequence): Print first names only

```
full_names = [
    "Ross Geller",
    "Rachel Green",
    "Phoebe Buffay",
    "Chandler Bing",
    "Monica Geller",
    "Joey Tribbiani"]
```




```
first_name_map = map(lambda x: x.split()[0], full_names)
print(type(first_name_map))
print(list(first_name_map))
```

Get the first element in the list

Map(function, sequence): Print first names only

```
full_names = [
    "Ross Geller",
    "Rachel Green",
    "Phoebe Buffay",
    "Chandler Bing",
    "Monica Geller",
    "Joey Tribbiani"]
```

Ross	Geller

Ross Geller
Rachel Green
Phoebe Buffay
Chandler Bing
Monica Gellar
Joey Tribbiani

```
first_name_map = map(lambda x: x.split()[0], full_names)
print(type(first_name_map))
print(list(first_name_map))
```

Split the string into a list of each word

Map(function, sequence): Print first names only

```
full_names = [
    "Ross Geller",
    "Rachel Green",
    "Phoebe Buffay",
    "Chandler Bing",
    "Monica Geller",
    "Joey Tribbiani"]
```

Ross	Geller

Ross Geller
Rachel Green
Phoebe Buffay
Chandler Bing
Monica Gellar
Joey Tribbiani

```
first_name_map = map(lambda x: x.split()[0], full_names)
print(type(first_name_map))
print(list(first_name_map))
```

Get the first element of that list of words and add it to the map

Map(function, sequence): Print first names only

```
full_names = [
    "Ross Geller",
    "Rachel Green",
    "Phoebe Buffay",
    "Chandler Bing",
    "Monica Geller",
    "Joey Tribbiani"]

# these two lines both extract the first names from full_names
first_name_map_list = list(map(lambda x: x.split()[0], full_names)) #map version

first_name_list_comprehension = [x.split()[0] for x in full_names] #comprehension version
```

#Notice **maps** don't need to use a ***for loop***. The **expression** is automatically applied to all members in the list

Map(function, seq_args_1, seq_args2, …, seq_args_n)

What if we have a lambda function that takes multiple arguments?

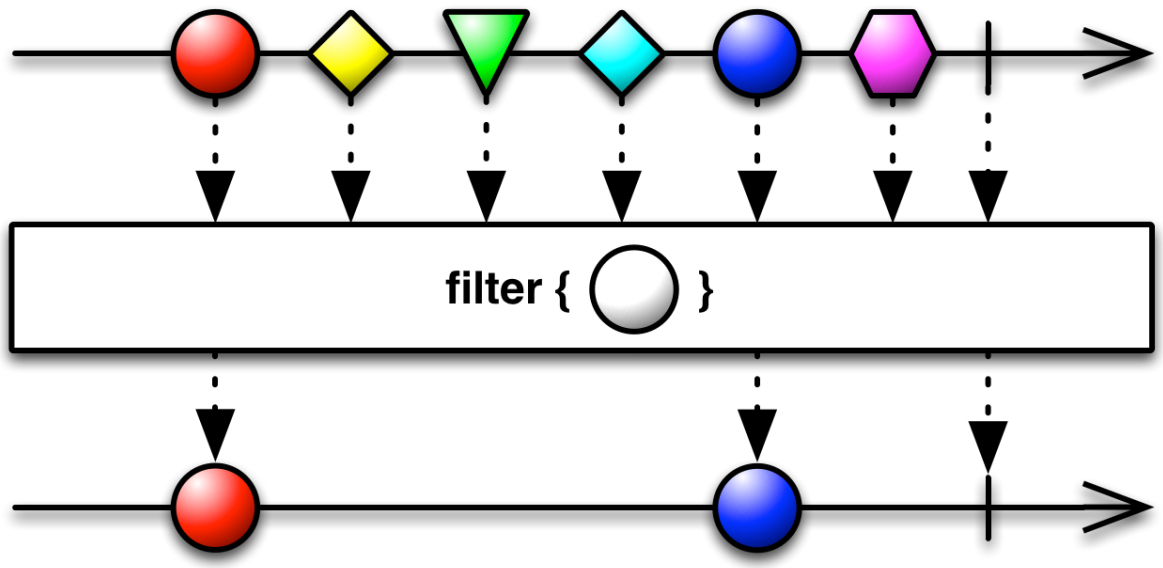
```
lambda x, y: f”{x} {y}”
```

```
first_names = ["Ross", "Rachel", "Pheobe", "Chandler", "Monica","Joey"]
last_names = ["Geller", "Green", "Buffay", "Bing", "Geller","Tribbiani"]

full_names_map = map(lambda x, y: f"{x} {y}", first_names, last_names)

print(type(full_names_map)) #<class 'map'>
print(tuple(full_names_map)) #('Ross Geller', 'Rachel Green', 'Pheobe Buffay', 'Chandler Bing',
'Monica Geller', 'Joey Tribbiani')
```

Filter

filter(function, sequence)

- Does exactly what the name suggests. Filters a number of elements from a given sequence.
- The first argument 'function' needs to be a function that returns True or False.
- This function is then applied to each element of the second argument, an iterable.
  - **filter(*function*, *iterable*)**
- If the result is false, that element is skipped. Otherwise it is included in the resultant filter object. This is an iterator similar to the map object.
- *filter()* can only work with **one argument** and **one sequence**.



Filter(function, sequence) – Example

Say we want to do the most cliché example.

Let's create a filter that extracts all the even numbers from a list with numbers 1 to 9.

Notice how the **lambda does not return the true/false value**

```
num_list = list(range(1, 10))
even_nums_filter = filter(lambda x: x % 2 == 0, num_list)
print(type(even_nums_filter)) #<class 'filter'>
print(list(even_nums_filter)) #[2, 4, 6, 8]
```

Filter(function, sequence) – Example

Say we want to do the most cliché example.

Let's create a filter that extracts all the even numbers from a list with numbers 1 to 9.

The same as before but using a **regular function** instead of *lambda*. Notice how we **must return the true/false value when using a function**

```
def even(x):
    return x % 2 == 0

num_list = list(range(1, 10))
even_nums_filter = filter(even, num_list)
print(type(even_nums_filter)) #<class 'filter'>
print(list(even_nums_filter)) #[2, 4, 6, 8]
```

Lambda Functions in use.

Challenge Time!!

Given a string “Hello world” and a list of vowels – [“a”, “e”, “i”, “o”, ”u”], write a filter that uses a lambda to select NON vowel characters.

```
string = "Hello world"
non_vowel_filter = ???
print(type(non_vowel_filter))
print(list(non_vowel_filter))
```

Lambda Functions in use.

Challenge Time!!

Given a string "Hello world" and a list of vowels – ["a", "e", "i", "o", "u"], write a filter that uses a lambda to select NON vowel characters.

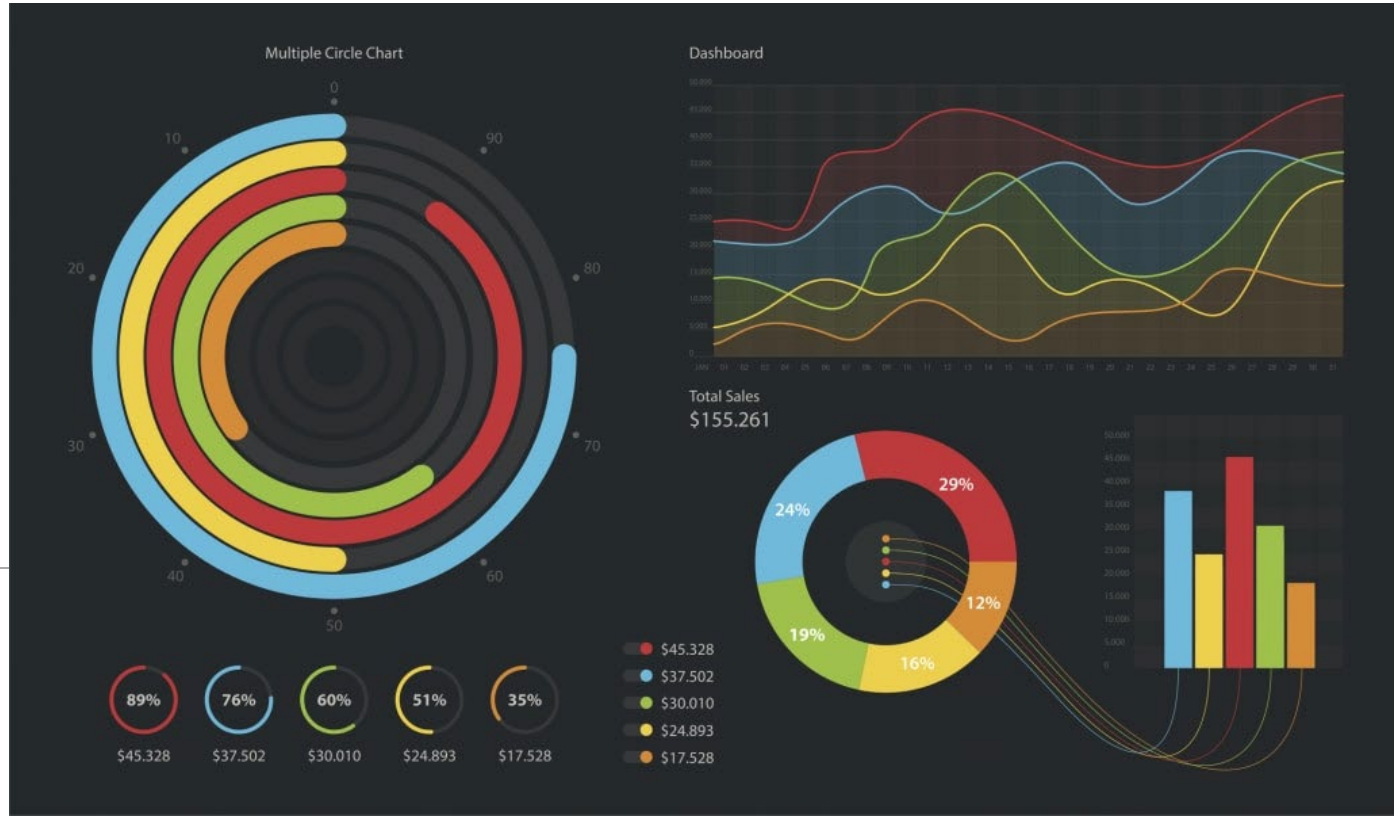
```
string = "Hello world"
non_vowel_filter = ???
print(type(non_vowel_filter))
print(list(non_vowel_filter))
```

```
string = "Hello world"
non_vowel_filter = filter(lambda x: x not in ['a','e','i','o','u'], string)
print(type(non_vowel_filter))
print(list(non_vowel_filter))
```

Profiling

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 6 PART 2



What is profiling?

Profiling is what we call analyzing our program's runtime performance:

- Space complexity
- Time complexity
- Usage of particular instructions
- Frequency and duration of function calls

Profiling is a form of dynamic program analysis

We build a profile with a **profiler**

We use the profile to aid in **program optimization**

Our goal is **correct, dependable, efficient (fast)** software.

Instrumentation

Profiling works by ***instrumenting*** the program source code or a binary executable

Instrumentation **adds code to a program to collection information about runtime behavior,** such as: function calls, memory usages, event frequency

Instrumentation incorporates (you don't need to memorize these fancy words):

- Code tracing
- Debugging
- Performance counters
- Logging, etc.

Profiler

We profile our code using a code **profiler**

Profilers use a variety of methods in addition to instrumentation:

- Events
- Statistics
- Simulations
- Hardware interrupts
- OS hooks
- Performance counters…

Profilers can produce **profiles**

A **profile** is a statistical summary of the events observed

The statistics are often displayed as annotations beside the source code, like this:

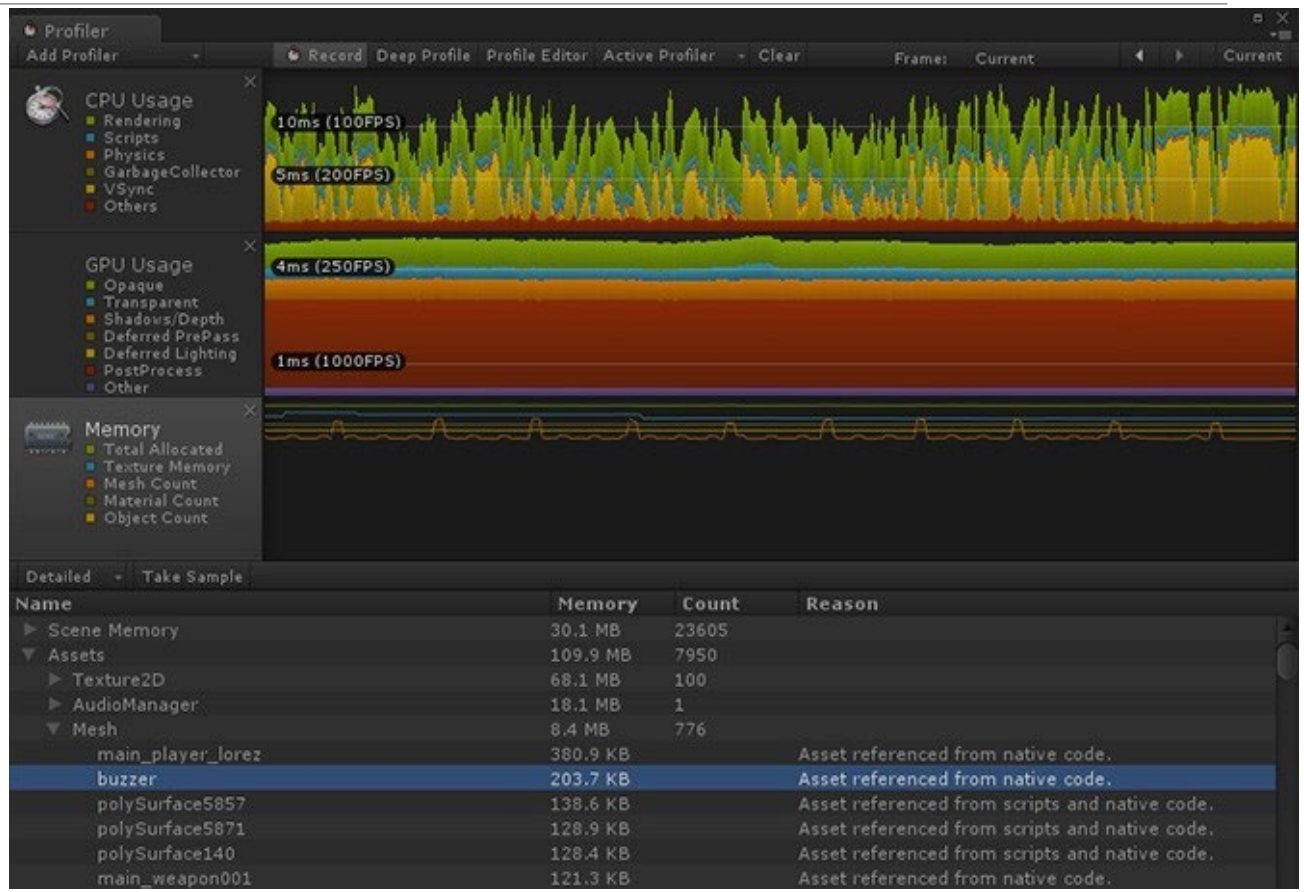
```
/* ------------- source------------------------------ count */
0001                IF X = "A"                          0055
0002                   THEN DO
0003                     ADD 1 to XCOUNT                0032
0004                   ELSE
0005                IF X = "B"                          0055
```

Profilers can produce **traces**

A **trace** is a stream of recorded events

We often use traces for parallel programs in order to understand how things are happening

A summary profile is usually sufficient for sequential programs



Unity Game Engine - Profiler

Building the profile

Profilers run during a program's execution (**dynamic analysis**)

The program execution must be interrupted by the profiler so that it can collect information

This can sometimes have a non-trivial effect on time measurements

We say that the **resolution** is "limited"

It's still helpful to us, though!

Ie:

2 seconds for code to run without profiler

2**.01** seconds for code to run WITH profiler

Profilers – 2 Types

Statistical Profiling

Deterministic Profiling

Statistical profiling

Statistical profiling randomly samples the instruction pointer and deduces where time is being spent

This involves less overhead because it doesn't require instrumentation

But it provides only *relative indications* of where the time was spent

The Python profiler does ***not*** use statistical profiling

Deterministic profiling

Deterministic profiling monitors all function calls, function returns, and exception events

Precise times are recorded for the intervals between these events

This requires overhead that can skew results. (Usually not a problem).

The Python profilers we can use are from the **cprofile** and **profile** modules

(Because there's a module for everything in Python, of course!)

Python's profilers use ***deterministic profiling***

How does Profiling help?

Call count statistics can be used to identify:

- Bugs (surprisingly high calls)
- Inline-expansion points
  places where we can reduce the overhead of calling functions and adding to the stack trace by replacing a function call with the body of the function call.

Internal time statistics can be used to identify "hot loops" that need to be optimized.
(This is a technical way of saying: identify functions that take too long and optimize them.)

Cumulative time statistics can be used to identify high-level errors in algorithms.
(This is a technical way of saying: find inefficient code logic spanning multiple functions and files and re-designing it.)

It all boils down to speed*. How fast can we make our program while ensuring it still works correctly?

* It's never, ever fast enough.

cProfile

cProfile.

Let’s profile something.

We can do it in 2 ways:

- Command Line (super useful)
- PyCharm (Professional Edition, available via education accounts)

cProfile – profiling a statement.

Open command line, type in the following

```
python
>>> import cProfile
>>> import re
>>> help(cProfile)
>>> help(re)
>>> help(cProfile.run)
```

cProfile – profiling a statement

Use cProfile to profile the compile function in the re module with parameter “foo|bar”

```
>>> cProfile.run('re.compile("foo|bar")')
```

cProfile results

Q: What's all this?

A: Stats from executing your program

Q: How are we supposed to read this?

A: I'll show you

Q: Can't we use PyCharm for this?

A: Yes can be done in command line or PyCharm

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

cProfile results

The first line indicates that **214 function calls were monitored**

Of those calls, **207 were *primitive***

Primitive calls are not recursive calls

```
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
```


```
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name
```

cProfile results

The next line tells us that the text string in the **far-right column** was used to **sort the output**

In this case the string is `filename:lineno(function)`

```
214 function calls (2

Ordered by: standard name
```

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        [illegible]    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

cProfile results

The first column heading is **ncalls**

This is (wait for it…) the **number of times the function was called**

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

ncalls t

cProfile results

The second column heading is **tottime**

This represents the **total time spent in the function**

It does not include time spent in calls to sub-functions

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

tottime
0.000

cProfile results

The third column heading is **percall**

percall = tottime / numcalls

We see that the **time per call** is negligible

(I expect library code to be highly optimized!)

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
   29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

percall

cProfile results

The fourth column heading is **cumtime** (cumulative time).

This is the **cumulative time spent in this and all subfunctions**

Counts from invocation till exit

This figure is accurate even for recursive functions.

cumtime p

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
        1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

cProfile results

The fifth column heading is **percall** again. Wait. What?

This is another call measure

This one **divides cumtime by primitive calls**

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        1    0.000    0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        1    0.000    0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        2    0.000    0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        1    0.000    0.000    0.000    0.000 sre_parse.py:76(__init__)
        2    0.000    0.000    0.000    0.000 sre_parse.py:81(groups)
        1    0.000    0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        1    0.000    0.000    0.000    0.000 sre_parse.py:919(parse)
        1    0.000    0.000    0.000    0.000 {built-in method _sre.compile}
        1    0.000    0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

```
percall
```

cProfile results

When there are **two values in the ncalls** column, it means the **function recursed**:

The **first number** is the **total number of calls**

The **second value** is the **number of primitive calls**

- Primitive calls do NOT include recursive calls

3/1

- **3** total calls
- **1** primitive call
- (**3**-**1**) = **2** recursive calls

```
>>> import cProfile
>>> import re
>>> cProfile.run('re.compile("foo|bar")')
         214 function calls (207 primitive calls) in 0.000 seconds

   Ordered by: standard name

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.000    0.000 <string>:1(<module>)
        2    0.000    0.000    0.000    0.000 enum.py:284(__call__)
        2    0.000    0.000    0.000    0.000 enum.py:526(__new__)
        1    0.000    0.000    0.000    0.000 enum.py:836(__and__)
        1    0.000    0.000    0.000    0.000 re.py:232(compile)
        1    0.000    0.000    0.000    0.000 re.py:271(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:249(_compile_charset)
        1    0.000    0.000    0.000    0.000 sre_compile.py:276(_optimize_charset)
        2    0.000    0.000    0.000    0.000 sre_compile.py:453(_get_iscased)
        [illegible]  0.000    0.000    0.000 sre_compile.py:461(_get_literal_prefix)
        [illegible]  0.000    0.000    0.000 sre_compile.py:492(_get_charset_prefix)
        [illegible]  0.000    0.000    0.000 sre_compile.py:536(_compile_info)
        [illegible]  0.000    0.000    0.000 sre_compile.py:595(isstring)
        1    0.000    0.000    0.000    0.000 sre_compile.py:598(_code)
      3/1    0.000    0.000    0.000    0.000 sre_compile.py:71(_compile)
        1    0.000    0.000    0.000    0.000 sre_compile.py:759(compile)
        3    0.000    0.000    0.000    0.000 sre_parse.py:111(__init__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:160(__len__)
       18    0.000    0.000    0.000    0.000 sre_parse.py:164(__getitem__)
        7    0.000    0.000    0.000    0.000 sre_parse.py:172(append)
      3/1    0.000    0.000    0.000    0.000 sre_parse.py:174(getwidth)
        1    0.000    0.000    0.000    0.000 sre_parse.py:224(__init__)
        8    0.000    0.000    0.000    0.000 sre_parse.py:233(__next)
        2    0.000    0.000    0.000    0.000 sre_parse.py:249(match)
        6    0.000    0.000    0.000    0.000 sre_parse.py:254(get)
        1    0.000    0.000    0.000    0.000 sre_parse.py:286(tell)
        1    0.000    0.000    0.000    0.000 sre_parse.py:417(_parse_sub)
        2    0.000    0.000    0.000    0.000 sre_parse.py:475(_parse)
        [illegible]  0.000    0.000    0.000 sre_parse.py:76(__init__)
        [illegible]  0.000    0.000    0.000 sre_parse.py:81(groups)
        [illegible]  0.000    0.000    0.000 sre_parse.py:903(fix_flags)
        [illegible]  0.000    0.000    0.000 sre_parse.py:919(parse)
        [illegible]  0.000    0.000    0.000 {built-in method _sre.compile}
        [illegible]  0.000    0.000    0.000 {built-in method builtins.exec}
       25    0.000    0.000    0.000    0.000 {built-in method builtins.isinstance}
    29/26    0.000    0.000    0.000    0.000 {built-in method builtins.len}
        2    0.000    0.000    0.000    0.000 {built-in method builtins.max}
        9    0.000    0.000    0.000    0.000 {built-in method builtins.min}
        6    0.000    0.000    0.000    0.000 {built-in method builtins.ord}
       48    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        5    0.000    0.000    0.000    0.000 {method 'find' of 'bytearray' objects}
        1    0.000    0.000    0.000    0.000 {method 'items' of 'dict' objects}

>>>
```

3/1

29/26

We can save the results to a file

Instead of printing the output, we can save it to a file

cProfile.run accepts an optional second parameter, a filename:

```
import cProfile
import re
cProfile.run('re.compile("foo|bar")', 'restats.txt’)
```

Open the file. What do you see?

profiling_example.py

This isn’t helpful, Jeff

Did you open the file?

It’s unintelligible

This needs translation

pstats.Stats (a statistics object) to the rescue

- Invoke its **constructor**
- Pass the **filename** as an argument to the constructor
- An optional second parameter, the output stream, defaults to sys.stdout

```
p = pstats.Stats(‘restats.txt’)
```

```
◆BEL7◆◆◆>g◆?◆e◆=ETX?0)ETXrVTNULNULNULi◆E
◆◆!◆>gGSFF◆DC1T◆◆>rESCNULNULNUL)EOTrETXN
NULNULNUL)ENQrBSNULNULNULrBSNULNULNULg◆[
◆◆!◆>{r+NULNULNUL)EOTrETXNULNULNULrETXNU
◆◆!◆>0)ETXrSOHNULNULNULi8STXNULNUL◆BSiss
◆◆!◆>g`EMeG◆|◆>{r*NULNULNUL)EOTrETXNULNU
◆◆!◆>0)ETXrVTNULNULNULr1NULNULNULr2NULNU
◆◆!◆>{rDC3NULNULNUL)EOTrNAKNULNULNULrNAK
◆◆!◆>0)ETXrVTNULNULNULiFSSOHNULNUL◆EOTte
◆◆!◆>gV◆◆◆◆◆1?{)ETX◆BS<string>rETXNULNUL
◆◆!◆>gV◆◆◆◆◆1?0)ETXrVTNULNULNUL◆0NULNULN
◆◆!◆>0)ETXrSOHNULNULNULi◆SOHNULNUL◆FF_ge
◆◆!◆>gw◆
◆◆!◆>{)ETXrSOHNULNULNULi◆SOHNULNULrSUBNU
◆◆!◆>gw◆
◆◆!◆>rSYNNULNULNUL)EOTrETXNULNULNULrETXN
◆◆!◆>gw◆
```

Try it!

```
import cProfile
import re
import pstats

cProfile.run(‘re.compile("foo|bar")’, ‘restats.txt’)
p = pstats.Stats(‘restats.txt’)
p.print_stats()
```

profiling_example.py

Whoops, I almost forgot…

The Stats class has a variety of methods for manipulating and printing the data saved in a profile results file

strip_dirs() removes the extraneous path from all the module names

sort_stats() sorts the entries according to the standard module/line/name string that is printed

```
from pstats import SortKey

p.strip_dirs().sort_stats(SortKey.STDNAME).print_stats()
```

* https://news.artnet.com/market/yoshitomo-nara-hong-kong-1671325

Sorting results

The **sort_stats( )** method accepts a string or a `SortKey` enum

The parameter identifies the basis of the sort

We can provide multiple keys that are applied in order.

Valid String Arg	Valid enum Arg	Meaning
`'calls'`	SortKey.CALLS	call count
`'cumulative'`	SortKey.CUMULATIVE	cumulative time
`'cumtime'`	N/A	cumulative time
`'file'`	N/A	file name
`'filename'`	SortKey.FILENAME	file name
`'module'`	N/A	file name
`'ncalls'`	N/A	call count
`'pcalls'`	SortKey.PCALLS	primitive call count
`'line'`	SortKey.LINE	line number
`'name'`	SortKey.NAME	function name
`'nfl'`	SortKey.NFL	name/file/line
`'stdname'`	SortKey.STDNAME	standard name
`'time'`	SortKey.TIME	internal time
`'tottime'`	N/A	internal time

Profiling a module

Profiling a module – Command line

This is the syntax from the command line:

```
python -m cProfile [-o output_file] [-s sort_order] filename.py
```

-m indicates profiling a module

-o is optional, if not provided the output will be displayed immediately.

-s is the field by which we can sort the results

Example: profiles **profile_ackermann.py** **sorted** by function **name**, output directly to console

```
python -m cProfile -s name profile_ackermann.py
```

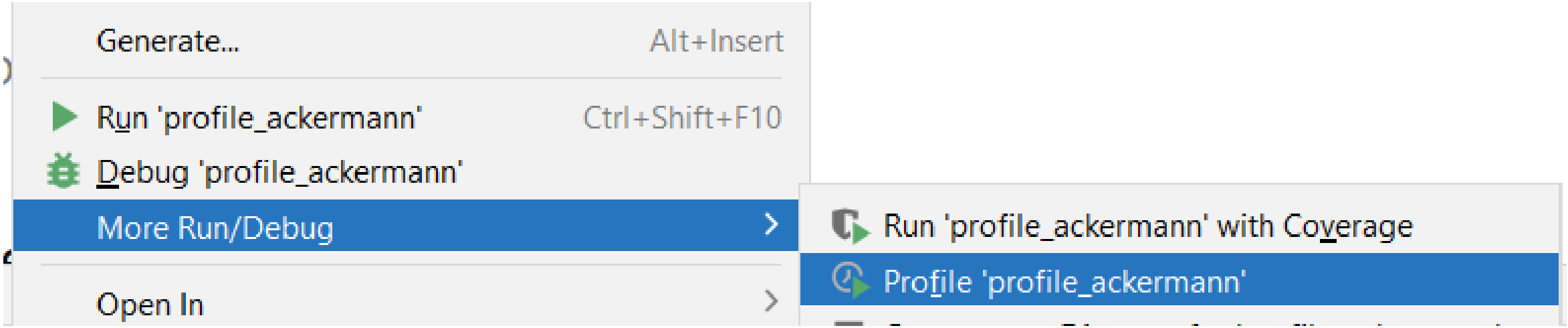
Profiling in Python using PyCharm

To profile in PyCharm:

- right click anywhere in your code

- hover “More Run/Debug”

- select “Profile ‘your module name’”



Profiling in Python using PyCharm

A new tab with the **statistics** of your code will appear

Select it to see stats including:

- Function name
- Call count
- Time (Time in function + children function time), Own time (Time in function)

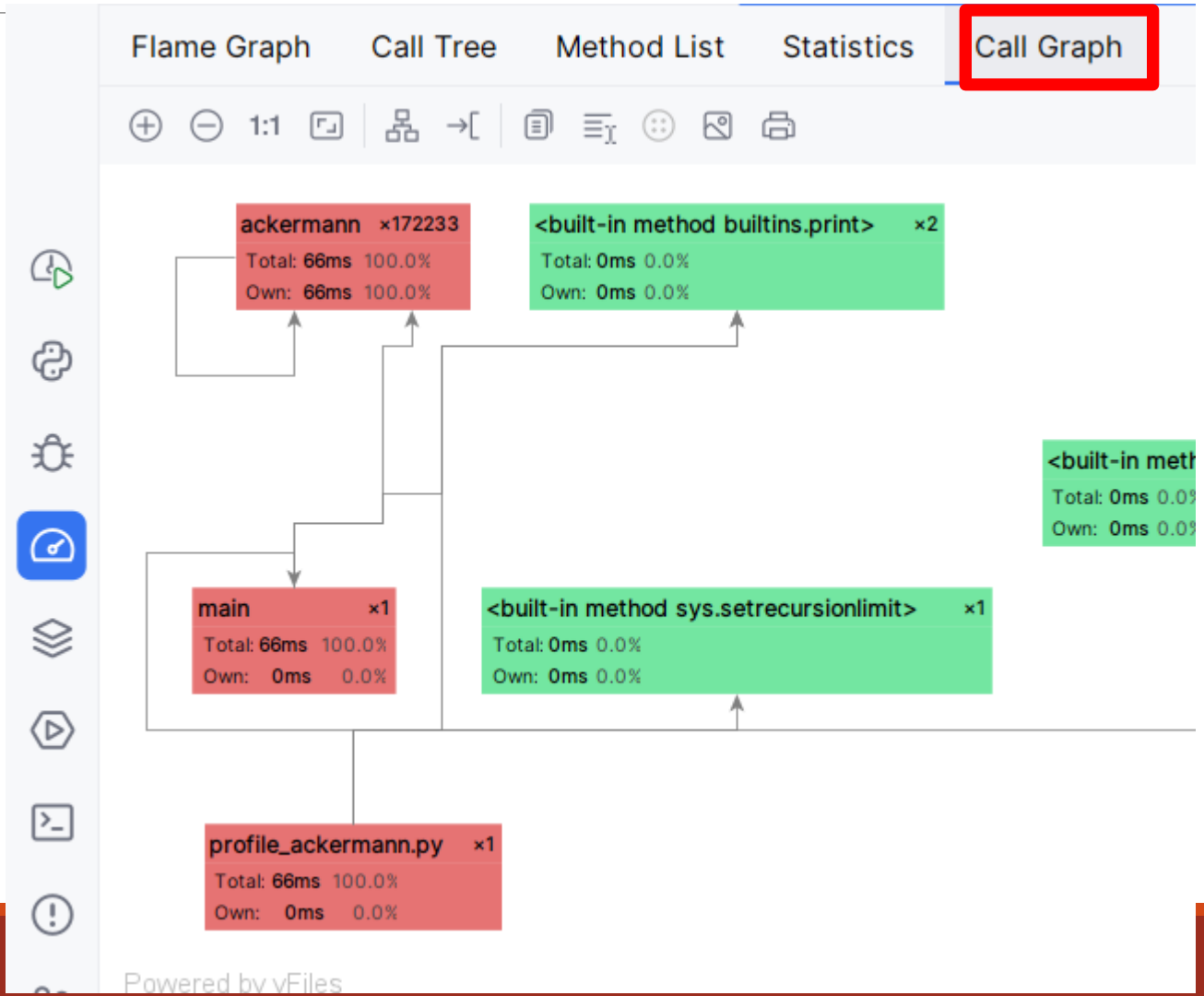
Name	Call Count	Time (ms)	Own Time (ms)
ackermann	172233	66 100.0%	66 100.0%
profile_ackermann.py	1	66 100.0%	0 0.0%
main	1	66 100.0%	0 0.0%
<built-in method builtins.print>	2	0 0.0%	0 0.0%
<built-in method sys.getrecursionlimit>	1	0 0.0%	0 0.0%
<built-in method sys.setrecursionlimit>	1	0 0.0%	0 0.0%

Profiling in Python using PyCharm

The **call graph** shows a visual representation of the calls made on your code

It includes the same information as the statistics tab

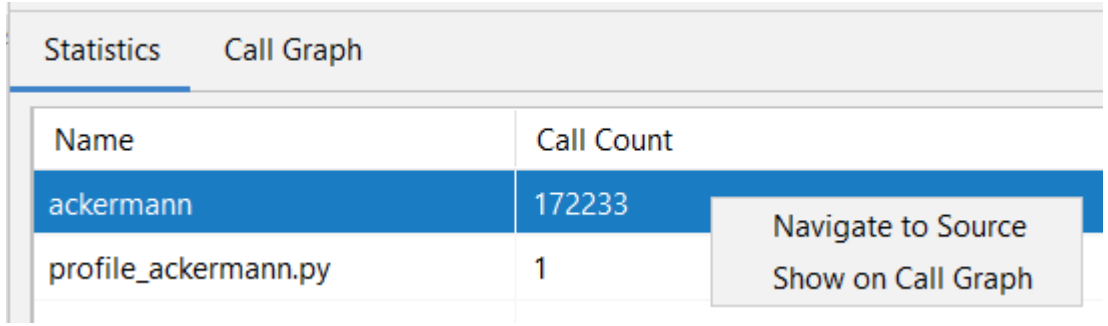
- Time
- Number of calls
- etc



Profiling in Python using PyCharm

To navigate to the **source code** of a certain function:

1. Right-click the corresponding entry on the Statistics tab
2. Choose Navigate to Source from the context menu



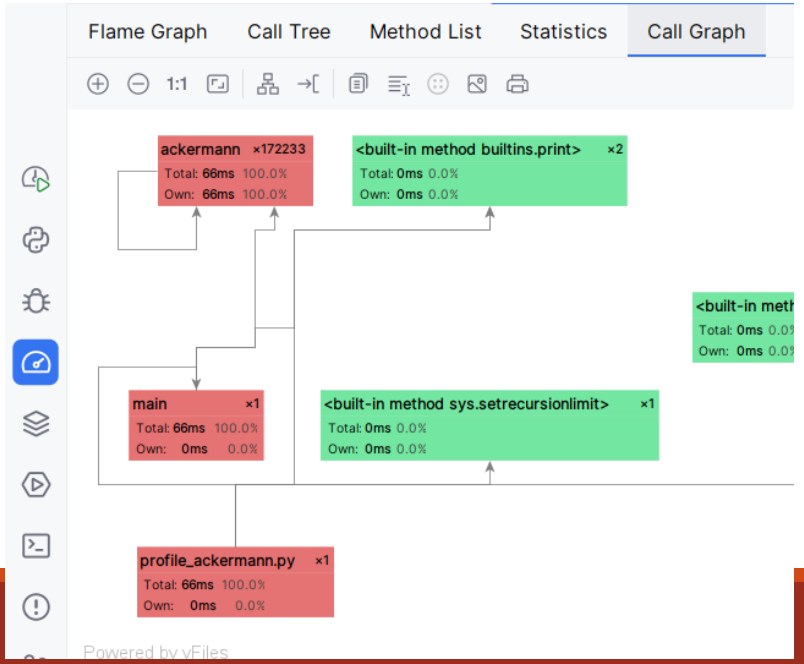
View the results: call graph

To navigate to the **call graph** of a certain function:

1. Right-click the corresponding entry on the Statistics tab
2. Choose Show on Call Graph from the context menu.
3. The Call Graph tab opens with the function in question highlighted

Note the color codes on the Call Graph

- The functions marked **red** consume more time
- The fastest functions are **green**



Challenge Time

Let's test a function called the Ackermann function

This recursive function takes a very long time. Start by invoking it with Ackermann(1, 2). Dare to try Ackermann(3, 7)

$$A(m,n) = \begin{cases} n+1 & \text{if } m = 0 \\ A(m-1,1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m-1, A(m, n-1)) & \text{if } m > 0 \text{ and } n > 0. \end{cases}$$

* https://en.wikipedia.org/wiki/Ackermann_function

Challenge Time

Let's test a function called the Ackermann function

This recursive function takes a very long time. Start by invoking it with Ackermann(1, 2). Dare to try Ackermann(3, 7)

```
import sys

def ackermann(m, n):
    if m == 0:
        return n + 1
    elif m > 0 and n == 0:
        return ackermann(m-1, 1)
    elif m > 0 and n > 0:
        return ackermann(m-1, ackermann(m, n-1))
```

```
def main():
    result = ackermann(3,7)
    print(result)
```

Start the profiling session

Do you blow your stack?

Modify your code to permit more recursive calls

The sys module has a handy dandy function for this

The highest possible limit is platform-dependent

This should be done with care, because an overly high limit can lead to a crash.

```
import sys
current = sys.getrecursionlimit() #For fun
sys.setrecursionlimit(10000)
```

Let’s try it out and discuss

Grab the code samples from today: profile_ackermann.py

Navigate to folder containing the python file:

Use cProfile from command line: `python -m cProfile -s name profile_ackermann.py`

How long did this take?

How many function calls took place?

- Primitive?
- Recursive?

How many different functions were invoked to execute your code?

What else did you see that looks helpful or ‘neat’?

Profiling a block of code

Profiling a block of code

```
import cProfile, pstats, io
from pstats import SortKey
pr = cProfile.Profile()
pr.enable()
# … code to be profiled comes here …
pr.disable()
sortby = SortKey.CUMULATIVE
ps = pstats.Stats(pr).sort_stats(sortby)
ps.print_stats()
```

profile_code_fragment.py, profile_code_fragment_2.py*

Want to learn more?

Check out the pstats module

https://docs.python.org/3.12/library/profile.html#module-pstats

The API is quite small and easy to learn

In fact, the API for cProfile is easy to learn

https://docs.python.org/3.12/library/profile.html#module-cProfile

Limitations: accuracy

There is a fundamental problem with profilers

- The underlying clock is only “ticking” once every millisecond
- No measurements can be more accurate than 0.001 seconds

There is also some lag:

- Delay from when an event is dispatched, until the profiler’s call to get the time actually *gets* the state of the clock
- Functions called many times, or which call many sub-functions, tend to accumulate this error
- The error is usually less than the accuracy of the tick (0.001 seconds) but it can accumulate and become significant